Data Models

SPARQL: Querying Knowledge Graphs

Tarapong Sreenuch

8 February 2026

克明峻德，格物致知

Building On: Topics 4 and 8

- **Topic 4:** SQL querying — FWGHOS pipeline, COUNT inflation from 1:many joins, LEFT JOIN preserving unmatched rows
- **Topic 8:** RDF knowledge graphs — triples, URIs, ontology, cross-source integration via shared identifiers, hand-traced traversals

Topic 8 left two questions open: how do we express hand-traced traversals as **queries a machine can execute**, and how do we **predict the result size** before running? This chapter closes both gaps — and the traps from Topic 4 transfer directly.

MovieGraph Analytics: The Shape IS the Query

An analyst asks: "Which person directed a sci-fi film that also won Best Picture?" She tries SQL — a 5-table join chain plus two WHERE clauses. 45 minutes to write and debug. MongoDB — no cross-collection pattern matcher. XPath — single tree, three sources.

The question is a **connected subgraph shape** — four nodes, three edges:

```
?person ──directed──→ ?film ──hasGenre──→ ex:genre_scifi
                        │
                        └──wonAward──→ ex:award_bestpicture
```

The shape itself is the query — but no existing language expresses "find this shape in a graph."

Hand-Tracing Does Not Scale

Topic 8's hand-tracing worked at 128 triples — 9 films, 3 directors.

MovieGraph then links to Wikidata: **2.1 million triples**, 350,000 films.

The analyst tries hand-tracing a 5-hop traversal: "actors who appeared in films by directors who also directed award-winning sci-fi films." After 90 minutes she has traced 40 of an estimated 12,000 candidate paths.

The traversal must become **declarative**: describe the shape, let the engine find it.

Multiplicity Hides in Patterns

A genre-diversity report: "How many films did each director direct?" The query includes a genre pattern to enrich the output.

Nolan's count: **9** instead of **6**.

```
6 films directed ──hasGenre──→ genres (1:many)

Batman Begins:        0 genres → DROPPED (inner join)
Dark Knight Rises:    0 genres → DROPPED
Inception:            2 genres → 2 bindings
Interstellar:         2 genres → 2 bindings
Oppenheimer:          2 genres → 2 bindings
The Dark Knight:      3 genres → 3 bindings
                                ──────────
                      Total:    9 bindings (not 6 films)
```

Same trap as Topic 4's COUNT inflation after joining through OrderLine — but with no column constraints to warn you.

The Pattern Across Three Failures

Failure	Root cause	What's missing
Shapes, not filters	Question is a subgraph, not a row/document/tree	A language for graph shapes
Hand-tracing doesn't scale	Manual traversal collapses at scale	Declarative execution
Multiplicity hides	1:many inflates, 0-match drops — silently	Binding-count prediction

Predict: Topic 4 introduced the predict-first discipline for SQL — predict row counts through each FWGHOS stage before running. Will the same discipline apply to graph queries? What would change?

Yes: Predict-First Transfers Directly

> Therefore, the engineer must **express graph traversals as declarative patterns and predict binding counts before running** — not discover after deployment that a 1:many pattern inflated results or an inner join silently dropped entities.

The mechanism changes (rows → bindings, tables → triple patterns), but the cognitive skill is identical. That transfer is the recurring concept across Topics 4, 6, 7, and now 9.

Five Capabilities the Failures Demand

#	Capability	Which failure it prevents
1	**Declarative pattern matching** — describe a subgraph shape, engine finds all matches	Failure 1: no language for graph shapes
2	**Optional connections** — LEFT JOIN semantics for missing data	Open-world: queries drop entities with missing data
3	**Aggregation over bindings** — COUNT, GROUP BY on matched patterns	Failure 3: wrong counts from multiplied bindings
4	**Multiplicity prediction** — trace binding count before running	Failure 3: invisible inflation
5	**Explicit variable scoping** — shared variable name = join key	Accidental joins from reused variable names

SPARQL — the SPARQL Protocol and RDF Query Language — provides all five by design.

SPARQL Vocabulary

Term	Definition
Triple pattern	A triple with one or more positions replaced by a `?variable`
Bindings table	The result of matching patterns — rows × variable columns
Basic graph pattern (BGP)	A set of triple patterns in `{ }` — the engine matches all simultaneously
Shared variable	A `?variable` appearing in multiple patterns — acts as an implicit join key
OPTIONAL	LEFT JOIN for patterns — preserves unmatched with unbound variables
FILTER	Boolean test applied after pattern matching — drops rows where expression is false or error

The Movie Knowledge Graph: Persons and Films

All queries run against the Topic 8 movie KG — 128 triples, 2 sources, 1 ontology.

URI	Name	Type(s)	Directed / ActedIn
ex:person_nm0634240	Christopher Nolan	Director	6 films
ex:person_nm0898288	Denis Villeneuve	Director	Dune
ex:person_nm1950086	Greta Gerwig	Director, Actor	Lady Bird, Barbie
ex:person_nm3154303	Timothée Chalamet	Actor	Interstellar, Dune
ex:person_nm0614165	Cillian Murphy	Actor	Dark Knight, Inception, Oppenheimer
ex:person_nm3053338	Margot Robbie	Actor	Barbie

The Movie Knowledge Graph: Films, Genres, Awards

Film	Year	Director	Genres (Source B)	Awards (Source B)
Inception	2010	Nolan	Sci-Fi, Action	—
Interstellar	2014	Nolan	Sci-Fi, Drama	—
Oppenheimer	2023	Nolan	Drama, Biography	Best Picture, Best Director, Best Actor
Batman Begins	2005	Nolan	— (no genre data)	—
The Dark Knight	2008	Nolan	Action, Crime, Drama	—
The Dark Knight Rises	2012	Nolan	— (no genre data)	—
Dune	2021	Villeneuve	Sci-Fi, Action, Drama	—
Lady Bird	2017	Gerwig	Comedy, Drama	—
Barbie	2023	Gerwig	Comedy	—

Key facts: Nolan has 6 films but only **4 have genre data**. Oppenheimer is the only film with awards (**3 awards**). Batman Begins and The Dark Knight Rises have **no genre triples** in Source B.

Cross-Source Integration: Where Sources Meet

```
Source A (filmography)                    Source B (genres & awards)

person_nm0634240                              genre_scifi
"C. Nolan"                                  "Science Fiction"
     │ directed                                  ▲ hasGenre
     ▼                                           │
┌──────────────────────────────────────────────────────────┐
│              film_tt1375666 ("Inception")                 │
│              ← same URI across both sources →             │
└──────────────────────────────────────────────────────────┘
     ▲ actedIn                                   │ hasGenre
     │                                           ▼
person_nm0614165                              genre_action
"C. Murphy"                                   "Action"
```

The film URI is the integration point — edges from different sources meet at the same node. A traversal from Nolan → Inception → Science Fiction **crosses the source boundary** at this URI.

Source	Triples	Provides
Ontology	34	6 classes, 9 properties
Source A	58	3 directors, 9 films, 3 actors, edges, titles, years
Source B	36	6 genres, 3 awards, classification edges, names

The Triple Pattern: SPARQL's Building Block

A **triple pattern** is a triple where one or more positions is a `?variable`. The engine matches it against every triple in the graph and returns one **binding** per match.

```
?film ex:hasGenre ex:genre_scifi .
```

"Which triples have predicate `ex:hasGenre` and object `ex:genre_scifi`?"

Against the movie KG, three films match:

?film
ex:film_tt1375666 (Inception)
ex:film_tt0816692 (Interstellar)
ex:film_tt1160419 (Dune)

Three triples matched → three bindings. Every SPARQL query builds on this: **patterns produce bindings tables**.

Bindings Are URIs, Not Readable Text

The bindings table contains opaque identifiers — `ex:film_tt1375666`, not "Inception."

To get human-readable output, add **label triples** to the pattern:

```
?film ex:hasGenre ex:genre_scifi .
?film ex:hasTitle ?filmTitle .
```

This ontology uses `ex:hasTitle` for films and `ex:hasName` for persons, genres, and awards. The universal standard is `rdfs:label`. Either way, the principle is the same: **identity (URI) and display name (literal) are separate triples** — just as Topic 2 separated primary keys from name columns.

The Basic Graph Pattern (BGP)

A **basic graph pattern** is a set of triple patterns in `{ }` within the WHERE clause. The engine matches all patterns simultaneously — only combinations where shared variables have consistent values survive.

```
PREFIX ex: <http://example.org/movie#>

SELECT ?filmTitle ?genreName
WHERE {
  ex:person_nm0634240 ex:directed ?film .
  ?film ex:hasTitle ?filmTitle .
  ?film ex:hasGenre ?genre .
  ?genre ex:hasName ?genreName .
}
```

Four patterns, one fixed node (Nolan), two output variables (?filmTitle, ?genreName), two intermediate join variables (?film, ?genre).

The BGP as a Graph Shape

```
ex:person_nm0634240 ──directed──→ ?film ──hasGenre──→ ?genre
      (Nolan)                       │                    │
                                    │                    │
                                 hasTitle             hasName
                                    │                    │
                                    ▼                    ▼
                                ?filmTitle           ?genreName
```

The BGP describes a **subgraph shape**. The engine finds every subgraph in the data that matches. Output variables (?filmTitle, ?genreName) carry display values. Intermediate variables (?film, ?genre) join patterns together.

Shared Variable = Join

When two patterns share a `?variable`, only combinations where that variable takes the **same value** in both patterns survive. The variable name IS the join key.

```
SQL:     customers c JOIN orders o  ON c.customer_id = o.customer_id
                                       ↑ explicit join key

SPARQL:  ex:person_nm0634240 ex:directed ?film .
         ?film ex:hasTitle ?filmTitle .
          ↑ shared variable ?film IS the join key
```

In SQL, join keys are explicit (ON clauses). In SPARQL, join keys are implicit — **shared variable names**. The mechanism is identical: restrict combinations to those where the shared column (SQL) or shared variable (SPARQL) agrees.

Step-by-Step Bindings Growth

Predict: The BGP from Slide 15 runs against the full movie KG. Will the result contain 6 rows (one per Nolan film), more, or fewer?

Bindings Growth: Traced

Step	Pattern Added	Bindings	Why
1	`Nolan ex:directed ?film .`	6	6 films directed by Nolan
2	`?film ex:hasTitle ?filmTitle .`	6	1:1 — one title per film
3	`?film ex:hasGenre ?genre .`	**9**	1:many — 2 films dropped (0 genres), 4 multiplied
4	`?genre ex:hasName ?genreName .`	9	1:1 — one name per genre

Step 3 is where both traps strike simultaneously:

- **Silent data loss:** Batman Begins and Dark Knight Rises dropped (0 genre matches — same as Topic 4's INNER JOIN)
- **Count inflation:** remaining 4 films multiplied by genre count: 2+2+2+3 = 9

Nine rows, 4 distinct films, 5 distinct genres. **COUNT(?film) returns 9 — not 6.**

The Multiplicity Rule

Each new pattern is a join. The relationship type determines what happens to the binding count:

Relationship	Example predicate	Effect on bindings	SQL parallel (Topic 4)
1:1	ex:hasTitle, ex:hasName	No growth	JOIN on unique FK
1:many	ex:hasGenre, ex:actedIn	Multiplication	JOIN to OrderLine
many:many	actors × films × genres	Potential explosion	JOIN through junction table

The practical test for each pattern you add: consult the entity reference or the property card. Ask: *"For each entity I already have bound, how many triples will this pattern match?"*

- `?film ex:hasTitle ?filmTitle` → exactly 1 per film → **1:1** → **safe**
- `?film ex:hasGenre ?genre` → 0 to 3 per film → **1:many** → **multiplication**

The Match-Set Model: Why Adding a Pattern Is Adding a Join

Each triple pattern independently produces a **match set** — one row per matching triple. The BGP joins these match sets on shared variables, exactly as SQL joins tables on shared columns.

```
Match set from Pattern 1:
?film
──────────────────────────
ex:film_tt1375666  (Inception)
ex:film_tt0816692  (Interstellar)
ex:film_tt15398776 (Oppenheimer)
ex:film_tt0468569  (The Dark Knight)
ex:film_tt0372784  (Batman Begins)
ex:film_tt1345836  (Dark Knight Rises)

6 rows
```

```
Match set from Pattern 3 (all films):
?film                 │ ?genre
──────────────────────┼────────────────────
ex:film_tt1375666     │ ex:genre_scifi
ex:film_tt1375666     │ ex:genre_action
ex:film_tt0816692     │ ex:genre_scifi
ex:film_tt0816692     │ ex:genre_drama
ex:film_tt15398776    │ ex:genre_drama
ex:film_tt15398776    │ ex:genre_biography
ex:film_tt0468569     │ ex:genre_action
ex:film_tt0468569     │ ex:genre_crime
ex:film_tt0468569     │ ex:genre_drama
ex:film_tt1160419     │ ex:genre_scifi
ex:film_tt1160419     │ ex:genre_action
ex:film_tt1160419     │ ex:genre_drama
ex:film_tt4925292     │ ex:genre_comedy
ex:film_tt4925292     │ ex:genre_drama
ex:film_tt1517268     │ ex:genre_comedy
15 rows
```

Join on `?film`: only Nolan's films **with genre data** survive → **9 rows**. Batman Begins (0 genre rows) and Dark Knight Rises (0 genre rows) are dropped — same as an INNER JOIN finding no matching rows.

The mental model: adding a pattern = adding a join. If the new match set has multiple rows per `?film` value (1:many), the join multiplies. If exactly one (1:1), the count is preserved.

OPTIONAL = LEFT JOIN

An inner pattern **drops** entities with no match. Batman Begins has no genre data → inner hasGenre pattern drops it entirely.

```
Inner pattern:    ?film ex:hasGenre ?genre .
  Batman Begins → 0 matches → DROPPED        (same as INNER JOIN)

OPTIONAL:         OPTIONAL { ?film ex:hasGenre ?genre . }
  Batman Begins → 0 matches → KEPT           (with ?genre unbound)
                                               same as LEFT JOIN → NULL
```

`OPTIONAL { pattern }` attempts to match. If no match exists, the variables remain **unbound** — not false, not empty string, genuinely unknown. This is SQL's LEFT JOIN producing NULL-padded rows.

OPTIONAL on Awards: Predict

Nolan directed 6 films. Oppenheimer has 3 awards. No other Nolan film has award data.

```
SELECT ?filmTitle ?awardName
WHERE {
  ex:person_nm0634240 ex:directed ?film .
  ?film ex:hasTitle ?filmTitle .
  OPTIONAL {
    ?film ex:wonAward ?award .
    ?award ex:hasName ?awardName .
  }
}
```

Predict: How many total rows? How many with ?awardName bound vs unbound?

OPTIONAL on Awards: Result

?filmTitle	?awardName
"Oppenheimer"	"Academy Award for Best Picture"
"Oppenheimer"	"Academy Award for Best Director"
"Oppenheimer"	"Academy Award for Best Actor"
"Inception"	_(unbound)_
"Interstellar"	_(unbound)_
"The Dark Knight"	_(unbound)_
"Batman Begins"	_(unbound)_
"The Dark Knight Rises"	_(unbound)_

8 rows: 3 from Oppenheimer's awards (1:many) + 5 unbound rows. The OPTIONAL preserved unmatched films — exactly as LEFT JOIN preserves unmatched rows in SQL.

The FILTER Trap: Three-Valued Logic

FILTER keeps bindings where the expression is **true** and drops everything else — including **errors**.

```
Bound value:   "Best Director" ≠ "Best Picture"  → true   → KEPT
Bound value:   "Best Picture"  ≠ "Best Picture"  → false  → DROPPED
Unbound value: (unbound)       ≠ "Best Picture"  → ERROR  → DROPPED
                                                    ↑
                                  Same as SQL: NULL = 'x' → UNKNOWN → dropped
```

Adding `FILTER(?awardName ≠ "Best Picture")` after the OPTIONAL:

- Oppenheimer / Best Director: true → **kept**
- Oppenheimer / Best Picture: false → dropped
- Inception (unbound): **error** → **dropped**
- All 5 unbound rows: **all dropped**

The FILTER obliterated every film without awards — the exact same trap as Topic 4's WHERE after LEFT JOIN.

FILTER Guard: Two Fixes

Fix 1 — Guard with !BOUND:

```
FILTER(!BOUND(?awardName) || ?awardName ≠ "Academy Award for Best Picture")
```

If unbound → keep. If bound → apply the comparison.

Fix 2 — Place FILTER inside OPTIONAL:

```
OPTIONAL {
  ?film ex:wonAward ?award .
  ?award ex:hasName ?awardName .
  FILTER(?awardName ≠ "Academy Award for Best Picture")
}
```

Same discipline as Topic 4: do not put right-table conditions in WHERE after LEFT JOIN. Put them in the ON clause (SQL) or inside OPTIONAL (SPARQL).

SQL LEFT JOIN vs SPARQL OPTIONAL: Parallel Behaviours

Behaviour	SQL (Topic 4)	SPARQL (Topic 9)
Preserve unmatched	LEFT JOIN → NULL-padded columns	OPTIONAL → unbound variables
Test for absence	IS NULL	!BOUND(?var)
Accidental filter	WHERE right_table.col drops NULLs	FILTER(?var = value) drops unbound
Safe filter placement	Put condition in ON clause	Put FILTER inside OPTIONAL { }

The mechanism is identical — same algebra, same trap, same fix. If you understood Topic 4's LEFT JOIN, you already understand OPTIONAL.

COUNT vs COUNT(DISTINCT): Aggregation

```
SELECT ?directorName (COUNT(DISTINCT ?film) AS ?filmCount)
WHERE {
  ?director rdf:type ex:Director .
  ?director ex:hasName ?directorName .
  ?director ex:directed ?film .
}
GROUP BY ?director ?directorName
ORDER BY DESC(?filmCount)
```

?directorName	?filmCount
Christopher Nolan	6
Greta Gerwig	2
Denis Villeneuve	1

Predict: If this query also included `?film ex:hasGenre ?genre .` in WHERE, what would Nolan's filmCount be with COUNT(?film) vs COUNT(DISTINCT ?film)?

COUNT Inflation: Both Traps Combined

Adding `?film ex:hasGenre ?genre .` as an inner pattern:

- Nolan's 4 genre-bearing films → 2+2+2+3 = **9 bindings**
- Batman Begins and Dark Knight Rises → **dropped** (0 genres)

Measure	Result	Problem
COUNT(?film)	**9**	Inflated by 1:many multiplication
COUNT(DISTINCT ?film)	**4**	Undercounts — 2 films silently dropped
True film count	**6**	Requires OPTIONAL + COUNT(DISTINCT)

The intersection of two Topic 4 traps: COUNT inflation **and** silent data loss. To get 6, you need OPTIONAL around the genre pattern and COUNT(DISTINCT ?film).

Property Paths: Multi-Hop Traversal

The sequel chain: Dark Knight Rises → Dark Knight → Batman Begins.

Path syntax	Meaning	Example
`ex:sequelOf`	Exactly 1 hop	Direct sequel
`ex:sequelOf/ex:sequelOf`	Exactly 2 hops	Sequel of a sequel
`ex:sequelOf+`	1 or more hops	Transitive closure
`ex:sequelOf*`	0 or more hops	Self plus all prequels
`ex:sequelOf{1,3}`	1 to 3 hops	Bounded depth
`^ex:sequelOf`	Inverse direction	Follow edge backwards

Predict: How many bindings does `ex:film_tt1345836 ex:sequelOf+ ?prequel .` produce?

Property Path: Traced

```
Start: ex:film_tt1345836 (The Dark Knight Rises)

Hop 1: ──sequelOf──→ ex:film_tt0468569 (The Dark Knight)    ✓ bound
Hop 2: ──sequelOf──→ ex:film_tt0372784 (Batman Begins)      ✓ bound
Hop 3: ──sequelOf──→ ???  (no outgoing edge)                 ✗ stops

Result: 2 bindings — The Dark Knight (1 hop), Batman Begins (2 hops).
```

With `*` instead of `+`: 3 bindings — includes Dark Knight Rises itself (zero hops). Usually not what you want.

Safety note: Prefer **bounded paths** (`{1,3}`) over unbounded (`+`, `*`). In a graph with cycles, an unbounded path can loop. Even without cycles, dense graphs can produce combinatorial explosions.

Query Anatomy: SPARQL vs SQL

```
PREFIX ex: <http://example.org/movie#>       -- 0. Namespace abbreviations

SELECT ?filmTitle ?genreName                  -- 4. Choose output variables
WHERE {                                       -- 1. Define patterns (BGP)
  ex:person_nm0634240 ex:directed ?film .
  ?film ex:hasTitle ?filmTitle .
  ?film ex:hasGenre ?genre .
  ?genre ex:hasName ?genreName .
  ?film ex:releaseYear ?year .
  FILTER(?year ≥ 2010)                        -- 2. Boolean filter
}
ORDER BY ?filmTitle                           -- 5. Sort bindings
LIMIT 10                                      -- 6. Truncate output
```

```
  SQL (fixed order)              SPARQL (flexible order)
  ─────────────────              ───────────────────────

  F: FROM/JOIN                   Patterns produce bindings
  W: WHERE                       FILTER narrows bindings
  G: GROUP BY                    GROUP BY partitions bindings
  H: HAVING                      HAVING filters groups
  O: ORDER BY                    ORDER BY sorts output
  S: SELECT                      SELECT chooses output variables
```

The logical model is the same. SQL guarantees fixed order; SPARQL's engine may reorder for optimisation.

SPARQL-to-SQL Concept Mapping

Concept	SQL (Topic 4)	SPARQL (Topic 9)
Data unit	Row in a table	Triple in a graph
Query unit	SELECT statement	SELECT with WHERE patterns
Result shape	Result set (rows × columns)	Bindings table (rows × variables)
Variable	Column selected/aliased	?variable in pattern
Join mechanism	JOIN...ON (shared column)	Shared variable across patterns
Preserved unmatched	LEFT JOIN (NULL-padded)	OPTIONAL (unbound variables)
Row/binding filter	WHERE clause	FILTER expression
Grouping	GROUP BY + aggregates	GROUP BY + aggregates
Duplicate control	DISTINCT, COUNT(DISTINCT)	DISTINCT, COUNT(DISTINCT)
World assumption	Closed	**Open**

The mechanisms are identical because SPARQL's designers built on the relational algebra deliberately.

End-to-End Worked Example: Nolan's Genres

```
Pipeline summary:

Pattern 1: Nolan directed ?film          6 rows   (base)
Pattern 2: ?film hasTitle ?filmTitle      6 rows   (1:1 — preserved)
Pattern 3: ?film hasGenre ?genre          9 rows   (1:many — 2 dropped, 4 multiplied)
Pattern 4: ?genre hasName ?genreName      9 rows   (1:1 — preserved)
                                          ──────
Final:                                    9 rows, 4 films, 5 genres
```

```
Compared to a naive prediction of "6 films, 6 rows":
✗ 2 films silently dropped  (Topic 4 trap: INNER JOIN)
✗ Count inflated 4→9        (Topic 4 trap: 1:many multiplication)

COUNT(?film) = 9   ← wrong (inflated)
COUNT(DISTINCT ?film) = 4   ← wrong (2 dropped)
True film count = 6   ← requires OPTIONAL + COUNT(DISTINCT)
```

This is the callback from Failure 3. The analyst got 9 instead of 6 because both traps struck simultaneously.

Three Divergences from SQL

Not everything transfers. Three differences matter for query design:

Divergence	SQL	SPARQL
Schema enforcement	DDL rejects bad inserts	Nothing rejects — ontology describes only
World assumption	Closed: "not in DB" = "not true"	Open: "not in graph" = "unknown"
Execution order	Fixed (FWGHOS)	Flexible — engine reorders for optimisation

Practical consequence: SPARQL queries must be **defensive** — patterns may match structurally unexpected triples. And `FILTER NOT EXISTS` means "no triples in THIS graph" — not "the relationship doesn't exist in the world."

The Seven-Step Query Procedure

Step	Action
1	**Identify starting entity and target** — known values → fixed URIs; unknown → ?variables
2	**Write the minimal BGP** — one pattern per edge in the desired shape
3	**Add one pattern at a time, predicting count** — 1:1 safe, 1:many multiplies
4	**Place FILTER with the trap in mind** — inside OPTIONAL or guard with !BOUND
5	**Add OPTIONAL last** — only for truly optional enrichment
6	**Decide if DISTINCT is needed** — any 1:many? Use COUNT(DISTINCT)
7	**Bounded property paths for multi-hop** — prefer `{1,N}` over unbounded `+`

For each step, predict the outcome before checking — same discipline as Topics 4 and 7.

Seven Steps Applied — Genre Diversity per Director

```
Step 1: Starting entity = all directors. Target = genre count.
Step 2: Minimal BGP:
  ?director rdf:type ex:Director .
  ?director ex:hasName ?directorName .
  ?director ex:directed ?film .
  ?film ex:hasGenre ?genre .

Step 3: Predict binding count:
  Pattern 1:  3    (Nolan, Villeneuve, Gerwig)
  Pattern 2:  3    (1:1 — one name per director)
  Pattern 3:  9    (1:many — Nolan 6, Villeneuve 1, Gerwig 2)
  Pattern 4:  15   (1:many — 2 films dropped (0 genres), rest by genre count:
                     Nolan: Inception(2)+Interstellar(2)+Oppenheimer(2)+Dark Knight(3)
                            +Batman Begins(0,dropped)+DK Rises(0,dropped) = 9
                     Villeneuve: Dune(3) = 3
                     Gerwig: Lady Bird(2)+Barbie(1) = 3.  Total: 9+3+3 = 15)
```

```
Step 6: Aggregation — genre is 1:many → use COUNT(DISTINCT ?genre)

Result:
directorName        | genreCount
--------------------+-----------
Christopher Nolan   | 5  (Sci-Fi, Action, Drama, Crime, Biography)
Denis Villeneuve    | 3  (Sci-Fi, Action, Drama)
Greta Gerwig        | 2  (Comedy, Drama)
```

Cost accepted: 2 of Nolan's films dropped (no genre data) — inner pattern trades completeness for simplicity.

Cross-Model Query Comparison

Dimension	SQL (Topic 2–5)	MongoDB (Topic 6)	XPath (Topic 7)	SPARQL (Topic 9)
Query unit	SELECT	find()/aggregate()	XPath expression	SELECT + WHERE
Atomic operation	Row filter + join	Document match	Node navigation	Pattern match
Join mechanism	JOIN...ON	$lookup	N/A (single tree)	Shared variable
LEFT JOIN	LEFT JOIN	$lookup default	N/A	OPTIONAL
Aggregation	GROUP BY	$group	count()	GROUP BY
Multiplicity	1:many joins	$unwind	Sibling count	1:many patterns
Ordering	ORDER BY	$sort	Document order	ORDER BY
World assumption	Closed	Closed	Closed	**Open**
Primary workload	Transactions	Aggregate read/write	Document exchange	Cross-source integration

The predict-first discipline applies across all four.

Library Knowledge Graph: A New Domain

Six checks against a **library KG** — different entities, different predicates, same mechanisms.

```
Authors:  lib:asimov   — "Isaac Asimov"    (wrote: Foundation, I Robot)
          lib:herbert  — "Frank Herbert"    (wrote: Dune)
          lib:bradbury — "Ray Bradbury"     (wrote: Fahrenheit 451; also lib:Editor)

Books:    lib:foundation  — "Foundation"           (genres: scifi, drama)
          lib:iRobot      — "I, Robot"             (genres: scifi)
          lib:duneNovel   — "Dune"                 (genres: scifi, adventure; prize:
Hugo)
          lib:fahrenheit  — "Fahrenheit 451"       (genres: scifi)
          lib:fAndE       — "Foundation and Empire" (sequelOf Foundation; no genres)
          lib:secondF     — "Second Foundation"     (sequelOf F&E; no genres)

Sequel:   secondF ──sequelOf──→ fAndE ──sequelOf──→ foundation
```

Check 1: Multiplicity Trace

A student queries "How many books did Asimov write?" with a BGP that includes a genre pattern:

```
SELECT (COUNT(?book) AS ?bookCount)
WHERE {
  lib:asimov lib:wrote ?book .
  ?book lib:hasGenre ?genre .
}
```

Asimov wrote 2 books. Foundation has 2 genres (scifi, drama). I, Robot has 1 genre (scifi).

> COUNT(?book) = **3** — wrong. Foundation produces 2 bindings (one per genre), I, Robot produces 1. The genre pattern multiplied bindings before COUNT saw them. **Fix:** COUNT(DISTINCT ?book) = 2. Same mechanism as Topic 4's COUNT inflation from joining through OrderLine.

Check 2: OPTIONAL + FILTER Trap

6 books. Only Dune has a prize. The student adds OPTIONAL for prize data, then filters **outside** the OPTIONAL:

```
OPTIONAL { ?book lib:wonPrize ?prize . ?prize lib:prizeName ?prizeName . }
FILTER(?prizeName ≠ "Hugo Award for Best Novel")
```

Intent: "Show books that didn't win the Hugo."

> **0 rows survive.** Dune's Hugo row: false → dropped. All 5 unbound rows: `unbound ≠ "Hugo … "` → **error** → **dropped**. The FILTER obliterated every book without prizes. **Fix:** Move FILTER inside OPTIONAL. Now the filter applies only within the optional block — Dune's Hugo row is filtered inside the OPTIONAL, so Dune appears with `?prizeName` unbound. All 6 books survive, all with unbound prize.

Check 3: FILTER NOT EXISTS (Anti-Join)

"Which books have no prize triples?"

```
SELECT ?title
WHERE {
  ?book rdf:type lib:Book .
  ?book lib:bookTitle ?title .
  FILTER NOT EXISTS { ?book lib:wonPrize ?prize }
}
```

Result: 5 books — everything except Dune.

> This is SPARQL's equivalent of SQL's left anti-join (`LEFT JOIN ... WHERE pk IS NULL`). **Open-world caveat:** The result means "no prize triples in THIS graph" — not "these books won no prizes." Foundation won the Hugo for Best All-Time Series in 1966, but this graph lacks that data.

Check 4: Shared Variable = Intersection

The library KG has 3 authors and 1 editor (Bradbury is both). A student wants "all authors and all editors":

```
?person rdf:type lib:Author .
?person rdf:type lib:Editor .
?person lib:authorName ?name .
```

Expected: 3 authors + 1 editor = 3 distinct persons. **Actual: 1** — only "Ray Bradbury."

```
Authors {Asimov, Herbert, Bradbury}     Editors {Bradbury}

        ┌──────────────────────┐
        │ Asimov    Herbert    │
        │           ┌──────────┼──────────┐
        │           │          │ Bradbury │
        └───────────┴──────────┴──────────┘
                  ↑ intersection = {Bradbury} ← shared ?person returns THIS
```

> **Intersection, not union.** The shared `?person` joins the two `rdf:type` patterns — only URIs matching BOTH survive. Fix: use UNION or `VALUES ?type { lib:Author lib:Editor }` to get all 3 distinct persons.

Check 5: Property Path vs Explicit Pattern

The sequel chain: Second Foundation → Foundation and Empire → Foundation.

```
-- Property path:
lib:secondF lib:sequelOf+ ?prequel .     -- 2 bindings (F&E, Foundation)

-- Explicit patterns with UNION:
{ lib:secondF lib:sequelOf ?prequel . }
UNION
{ lib:secondF lib:sequelOf/lib:sequelOf ?prequel . }
```

Both return 2 results. With `*` instead of `+`: 3 — includes Second Foundation itself (zero hops).

> **Property paths** are concise and extend to deeper chains automatically. **Explicit patterns** are more transparent and allow filtering intermediate hops — which SPARQL property paths cannot do. Use paths when depth varies; use explicit patterns when intermediate filtering is needed.

Check 6: SQL LEFT JOIN vs SPARQL OPTIONAL

6 books. Only Dune has a prize.

```
SQL LEFT JOIN result:
title                  │ prize_name
──────────────────────┼──────────────
"Foundation"           │ NULL
"I, Robot"             │ NULL
"Dune"                 │ "Hugo … "
"Fahrenheit 451"       │ NULL
"Foundation & Empire"  │ NULL
"Second Foundation"    │ NULL
── 6 rows ──

SPARQL OPTIONAL result:
?title                 │ ?prizeName
──────────────────────┼──────────────────
"Foundation"           │ (unbound)
"I, Robot"             │ (unbound)
"Dune"                 │ "Hugo Award … "
"Fahrenheit 451"       │ (unbound)
"Foundation & Empire"  │ (unbound)
"Second Foundation"    │ (unbound)
── 6 rows ──
```

> Both produce 6 rows — identical structure, identical trap, identical fix. **SQL:** `WHERE prize_name = 'x'` drops NULLs. **SPARQL:** `FILTER(?prizeName = "x")` drops unbound. The predict-first discipline transfers directly.

Earlier We Asked: Predict-First Transfers?

In Slide 6, we asked: will the predict-first discipline from Topic 4's SQL apply to graph queries?

The answer across six scenarios: **yes — the cognitive skill is identical.** Multiplicity prediction (Checks 1, 5), the FILTER trap (Checks 2, 3), and join semantics (Checks 4, 6) all trace to the same root cause as Topic 4 — not understanding what happens to the binding count when a pattern is added.

The mechanism changed (rows → bindings, tables → triple patterns). The discipline did not.

What SPARQL Does NOT Solve

Limitation	What SPARQL lacks	Better tool	Topic
Per-hop edge filtering	Property paths can't filter intermediate nodes	Cypher (variable-length + WHERE)	10
Cycle detection	No native cycle detection; unbounded paths may loop	Cypher (cycle-aware traversal)	10
Transactional workload	No ACID write transactions across triples	Relational engine	5
Single-source property graph	URI alignment unnecessary; traversal-heavy with edge labels	Cypher	10

Cost accepted: SPARQL excels at cross-source integration via URI alignment. For single-source traversal, Cypher is more natural.

Concepts Planted Here That Recur

Concept	Introduced here as	Recurs in
Bindings table	Pattern match results as rows × variable columns	Topic 10 (Cypher result tables), Topic 11 (cross-model comparison)
Shared variable = join	Implicit join via variable name reuse	Topic 10 (Cypher pattern matching)
OPTIONAL = LEFT JOIN	Preserve unmatched with unbound variables	Topic 11 (query equivalence across models)
Multiplicity prediction	Trace bindings growth before running	Topic 10 (path count prediction)
FILTER NOT EXISTS	Anti-join for graph patterns	Topic 10 (WHERE NOT in Cypher)

Course Map

```
Part 1 — Relational Foundation
├── Ch 1  Introduction to Data Models
├── Ch 2  Conceptual Modelling (ERD)
├── Ch 3  Normalisation (BCNF & Star Schemas)
├── Ch 4  SQL Querying (FWGHOS Pipeline)
└── Ch 5  Enforcement (Constraints, Access Control, Transactions, Indexes)

Part 2 — Alternative Models
├── Ch 6  Document Model (JSON & MongoDB)
├── Ch 7  XML for Contracts (XSD & XPath)
├── Ch 8  RDF and Knowledge Graphs
├── Ch 9  SPARQL Querying  ◄── you are here
└── Ch 10 Property Graphs (Cypher)

Part 3 — Capstone
└── Ch 11 Multi-Model Design
```

Topic 9 formalises Topic 8's hand-traced traversals as SPARQL queries. Topic 10 introduces Cypher for property graphs — same movie domain, per-hop filtering that SPARQL lacks. Topic 11 assigns SPARQL to the cross-source integration workload in the capstone architecture.

Lab Preview

Lab 9 — SPARQL: Querying Knowledge Graphs

A code-based lab using SPARQL queries against the movie knowledge graph.

- **Parts 1–2** (~30 min): Manual bindings table tracing on paper — predict before executing
- **Parts 3–5** (~60 min): Executable SPARQL queries using `%%sparql` magic — pattern matching, OPTIONAL, FILTER, aggregation
- **Parts 6–7** (~30 min): Property paths and graph mutation with rdflib

Format: Query-execution lab in Google Colab. Predict-first discipline throughout. **Total ~120 min.**

File: `09-lab-sparql-querying-knowledge-graphs.ipynb`